ARCA Ltda. — Sprint Backlog del Sprint 1

Sistema de Gestión Humana · Arquitectura de Software (SIS-311) · Universidad Católica Boliviana "San Pablo" · Gestión 2026

1. Sprint Backlog - Sprint 1

Objetivo del sprint: dejar operativas las fundaciones técnicas del sistema (monorepo, bases de datos por servicio, mensajería con RabbitMQ, identidad con JWT y control de acceso por rol) y entregar los módulos de Personal y Vacaciones de punta a punta, incluyendo el bloqueo de bajas por inamovilidad, el flujo de aprobación de vacaciones en 2 niveles y la auditoría inmutable, para alcanzar la meta de ponderación del 70%.

Duración: 27 de octubre al 3 de noviembre de 2026 (1 semana).

Story points comprometidos: 54 SP (HU-01: 3, HU-02: 3, HU-03: 3, HU-04: 3, HU-05: 5, HU-06: 3, HU-07: 8, HU-08: 2, HU-10: 3, HU-11: 3, HU-12: 3, HU-13: 5, HU-14: 5, HT-04: 5).

Equipo y roles del Sprint 1

Rol	Responsable
Arquitecto de Software y encargado de la base de datos	Axl Severich
Scrum Master (además backend de vacaciones)	Jean Fernandez
Dev Frontend	Luciana Yahuita
Dev Backend (identidad y catálogos)	Laura Pérez
Dev Backend (personal)	Elias Lecoña

Tarea	Historia / Origen	Descripción / Definición de Hecho (DoD)	Responsable
TSK-ARQ-101: Monorepo, esqueleto hexagonal de los servicios y API Gateway base	Setup / RNF-08 / RNF-04	• Crear el monorepo (pnpm workspaces) con apps/web-frontend, apps/api-gateway, services/ms-auth, ms-personal, ms-vacaciones, ms-contratos, ms-planillas y packages/ (common-dtos, crypto-utils). • Plantilla única de servicio NestJS con capas Domain / Application / Infrastructure, endpoint /health y Swagger en /docs. • API Gateway: enrutamiento a los servicios y verificación de JWT con la clave pública RS256 (en pareja con Laura, TSK-DEV2-103). • Convenciones: Conventional Commits, ramas feature/<ID>-descripcion, main protegida, PR con al menos una revisión, ESLint y Prettier.	Arquitecto (Axl Severich) + Dev Backend (Laura Pérez)

Tarea	Historia / Origen	Descripción / Definición de Hecho (DoD)	Responsable
		DoD: Los 5 servicios y el gateway levantan con /health en verde; el repositorio tiene README con instrucciones de arranque; regla de dependencias activa en CI (el Domain no importa Infrastructure).	
TSK-ARQ-102: Diseño e implementación de las 5 bases de datos PostgreSQL	Setup / RNF-05 / RNF-06 / RNF-07	• Scripts de inicialización de db_auth, db_personal, db_vacaciones, db_contratos y db_planillas (una BD por servicio), con un schema.prisma y migraciones por servicio. • Modelo de datos de Sprint 1: usuario, rol, permiso, auditoria_log (db_auth); empleado, area, cargo, historial_laboral, inamovilidad_laboral, baja (db_personal); saldo_vacaciones, solicitud_vacacion, feriado, empleado_ref (db_vacaciones). • Cifrado AES-256 de campos sensibles (CI y remuneración) mediante utilidad compartida en packages/crypto-utils; tabla auditoria_log de solo inserción (sin permisos de UPDATE ni DELETE). • Datos semilla: 5 roles con su matriz de permisos, usuario ADMIN_GENERAL inicial, feriados nacionales 2026 y catálogo base de áreas y cargos. • Modelo inicial de db_planillas (parametro_global, planilla_mensual, boleta_pago) como parte del spike de boletas (TSK-ARQ-108). **DoD:** Migraciones aplicadas desde cero con un solo comando; semillas cargadas; diccionario de datos por servicio publicado en docs/db/ y MER exportado; ningún servicio tiene credenciales de la BD de otro.	Arquitecto (Axl Severich)
TSK-ARQ-103: Docker Compose del entorno local	Setup / RNF-09	• docker-compose.yml con PostgreSQL (5 bases), RabbitMQ con consola de administración, los 5 servicios, el gateway y el frontend. Dockerfile genérico para servicios NestJS y otro para Next.js. • .env.example con todas las variables (puertos, claves JWT, Cloudinary, RabbitMQ) sin valores reales. **DoD:** Un integrante nuevo levanta todo el sistema con docker compose up en menos de 10 minutos siguiendo el README.	Arquitecto (Axl Severich)
TSK-ARQ-104: Contratos de API y de eventos del Sprint 1	HU-01 a HU-14 / RF-13 / RF-30	• Documentar en docs/api/ los contratos de auth.md (login, refresh, usuarios, roles, auditoría), personal.md (funcionarios, áreas, cargos, baja, inamovilidad) y vacaciones.md (saldos, solicitudes, aprobación, excepción), con códigos de error comunes (docs/api/errores.md). • Definir docs/api/eventos.md: EMPLEADO_CREADO, EMPLEADO_MODIFICADO, EMPLEADO_DADO_DE_BAJA y AUDITORIA_REGISTRADA, con payload versionado (eventId, eventName, occurredOn, payload). **DoD:** Contratos publicados y aprobados por Laura, Elias, Jean y Luciana hasta el 30/10, para que backend y frontend trabajen en paralelo; cualquier cambio posterior pasa por PR documentado.	Arquitecto (Axl Severich) + Dev Backend (Laura Pérez) + Dev Backend (Elias Lecoña) + Scrum Master/Backend (Jean Fernandez)

Tarea	Historia / Origen	Descripción / Definición de Hecho (DoD)	Responsable
TSK-ARQ-105: HT-04 Sincronización de eventos con RabbitMQ	HT-04 (5 SP) / RF-13 / RF-30 / RNF-08	• Exchange de tipo topic, colas durables por servicio consumidor y DLQ con reintentos con retraso. • Librería compartida en packages/ para publicar (patrón Outbox: el evento se guarda en la misma transacción del cambio de estado) y consumir con idempotencia por eventId. • Primer flujo de extremo a extremo: EMPLEADO_CREADO (ms-personal) crea el saldo inicial en ms-vacaciones; AUDITORIA_REGISTRADA (todos los servicios) llega a ms-auth. **DoD:** Un evento publicado por ms-personal es consumido por ms-vacaciones y ms-auth; si el consumidor cae, el mensaje se reintenta y no se pierde; los mensajes fallidos terminan en la DLQ visible en la consola de RabbitMQ. Pruebas de integración en verde.	Arquitecto (Axl Severich)
TSK-ARQ-106: Diagramas C4 y diagramas de secuencia	Entregable de arquitectura / RNF-08	• C4 nivel 1 (contexto con el Core de ARCA y el Directorio), nivel 2 (contenedores) y nivel 3 (componentes de Personal y Vacaciones). • Diagramas de secuencia: alta de funcionario hasta la creación del saldo de vacaciones, y solicitud de vacaciones con su aprobación en 2 niveles y la ruta de excepción. **DoD:** Diagramas publicados en docs/ (draw.io y Mermaid), enlazados en el README y explicados en el Sprint Review.	Arquitecto (Axl Severich)
TSK-ARQ-107: ADR-001 a ADR-004	Entregable de arquitectura / RNF-04 / RNF-05 / RNF-08	• ADR-001: microservicios con base de datos por servicio y arquitectura hexagonal ligera. • ADR-002: mensajería asíncrona con RabbitMQ, patrón Outbox y consumidores idempotentes. • ADR-003: autenticación con JWT RS256, refresh tokens, RBAC de 5 roles y auditoría inmutable. • ADR-004: regla de vacaciones del MVP (15 días por año cumplido) frente a la escala LGT parametrizable, y reglas de inamovilidad. **DoD:** ADRs en docs/adr/ y documento Word con la justificación en la carpeta de entregables.	Arquitecto (Axl Severich)
TSK-ARQ-108: Spike de cálculo de boletas y validación de RNF / code review	Spike Sprint 2 / RNF-01 / RNF-04 / RNF-05	• Spike: probar con casos calculados a mano el Total Ganado, Bono de Antigüedad, aportes de 12.71%, ANS y RC-IVA, y dejar el resultado en docs/spikes/boletas.md para el Sprint 2. • Prueba de latencia (autocannon o k6) de POST /auth/login y GET /employees: p95 menor a 300 ms. • Verificar el aislamiento: ningún servicio lee la BD de otro y los campos AES-256 no aparecen en claro en la base ni en los logs. • Code review de todos los PR que toquen auth, gateway, eventos y esquemas de base de datos. **DoD:** Reporte con p95 menor a 300 ms; spike documentado con los casos de prueba; PR críticos revisados.	Arquitecto (Axl Severich)
TSK-DEV1-101: Proyecto Next.js, sistema de diseño y layout base	Setup / RNF-10 / RNF-01	• Next.js (App Router) con Tailwind CSS y shadcn/ui; layout con sidebar y header, menú según el rol, modo claro/oscuro, componentes compartidos (tablas, formularios, diálogos, ErrorAlert, estados de carga y vacío) y cliente API con TanStack Query.	Dev Frontend (Luciana Yahuita)

Tarea	Historia / Origen	Descripción / Definición de Hecho (DoD)	Responsable
		• Guía de estilos y prototipo de las vistas clave en Figma o Penpot (paleta slate-900 / blue-600, tipografía Inter, iconos Lucide). **DoD:** Layout responsive navegable con rutas vacías de los 4 módulos; guía de estilos y prototipo aprobados por el equipo; las pantallas cargan en menos de 1.5 s.	
TSK-DEV1-102: Login, sesión y control de acceso por rol en el frontend	HU-01 / HU-03 / RF-01 / RF-02 / RF-03	• Pantalla de login con validación y mensajes de error claros, manejo de sesión (access y refresh token), cierre de sesión y renovación automática. • Guard de rutas y ocultamiento de opciones del menú según la matriz RBAC; redirección con mensaje cuando el rol no tiene acceso. **DoD:** Un usuario de cada rol inicia sesión y solo ve sus opciones; al expirar el token se renueva o se redirige al login; mocks usados mientras el backend de Laura no esté disponible.	Dev Frontend (Luciana Yahuita)
TSK-DEV1-103: Pantallas de usuarios y roles, y visor de auditoría	HU-02 / HU-04 / RF-04 / RF-05	• Administración de usuarios: lista, creación, activación o bloqueo y asignación de rol (solo ADMIN_GENERAL). • Visor de auditoría de solo lectura con filtros por usuario, módulo y fecha, y detalle del payload previo y posterior. **DoD:** Pantallas conectadas a los endpoints de Laura; el visor no ofrece ninguna acción de edición ni eliminación; revisadas en el Sprint Review.	Dev Frontend (Luciana Yahuita)
TSK-DEV1-104: Búsqueda y listado de personal	HU-08 (2 SP) / RF-09	• Tabla de funcionarios con búsqueda en tiempo real por nombre o CI, filtros por área, cargo y estado (EN_PRUEBA, RATIFICADO, INACTIVO), paginación y badges de estado e inamovilidad. Datos con TanStack Query, estados de carga y de error. **DoD:** Filtros combinados funcionando contra GET /employees de Elias; la tabla responde sin recargar la página.	Dev Frontend (Luciana Yahuita)
TSK-DEV1-105: Formularios de alta, modificación y baja de funcionarios	HU-05 / HU-06 / HU-07 / RF-06 / RF-07 / RF-08	• Formulario de alta con datos personales, área, cargo, haber básico, fecha de ingreso, carga de la firma gráfica (.png/.jpeg) y marcas de inamovilidad. Formulario de modificación con fecha efectiva y motivo, y acción de ratificación. • Flujo de baja: al intentar dar de baja a un funcionario con inamovilidad activa, un diálogo de alerta bloquea la acción y solicita la autorización del Ministerio de Trabajo; si procede, muestra la liquidación (finiquito) antes de confirmar. **DoD:** Alta, modificación, ratificación y baja funcionando de punta a punta contra el backend de Elias; el bloqueo por inamovilidad se demuestra en la demo.	Dev Frontend (Luciana Yahuita)
TSK-DEV1-106: Organigrama interactivo y administración de áreas y cargos	HU-10 (3 SP) / HU-11 / RF-10 / RF-12	• Organigrama en árbol dinámico con áreas, cargos, integrantes y jefe de cada área. • Pantalla de administración de áreas (jerarquía padre/hija) y cargos, con bloqueo de eliminación cuando hay funcionarios asignados. **DoD:** El organigrama se genera a partir de los datos reales de ms-personal y se actualiza al crear un área o cargo; revisado en el Sprint Review.	Dev Frontend (Luciana Yahuita)

Tarea	Historia / Origen	Descripción / Definición de Hecho (DoD)	Responsable
TSK-DEV1-107: Pantallas de vacaciones y flujo de aprobación	HU-12 / HU-13 / HU-14 / RF-14 / RF-15 / RF-16 / RF-17	• Vista del saldo por gestión (días asignados, tomados y pendientes) y formulario de solicitud con selección de rango de fechas, cálculo de días hábiles y alertas de traslape o saldo insuficiente. • Bandeja de aprobación para Jefe de Área y RRHH con estados PENDIENTE_JEFE, APROBADO_JEFE y APROBADO_FINAL. Cuando se exceden los días disponibles, se habilita el botón "Solicitar aprobación excepcional" dirigido a Gerencia. **DoD:** Un funcionario solicita, el jefe aprueba, RRHH aprueba y el saldo se actualiza en pantalla; la ruta de excepción se demuestra con una solicitud que excede el saldo.	Dev Frontend (Luciana Yahuita)
TSK-DEV2-101: ms-auth: inicio de sesión con JWT RS256 y refresh tokens	HU-01 (3 SP) / RF-01 / RF-02 / RNF-04	• POST /auth/login valida credenciales con Argon2id y emite un access token RS256 (TTL de 15 min) con id_usuario, codigo_rol y permisos, más un refresh token cifrado en base de datos (TTL de 8 h). • POST /auth/refresh y POST /auth/logout (invalida el refresh token). Las claves RS256 se cargan desde variables de entorno. **DoD:** Login correcto devuelve ambos tokens; credenciales inválidas devuelven un error genérico; los tokens expirados se rechazan. Pruebas unitarias e integración en verde.	Dev Backend (Laura Pérez)
TSK-DEV2-102: ms-auth: gestión de usuarios y roles	HU-02 (3 SP) / RF-04	• CRUD de usuarios (crear, activar, bloquear, inactivar) y asignación de rol, solo para ADMIN_GENERAL. • Bloqueo automático al tercer intento fallido (MAX_FAILED_LOGIN_ATTEMPTS) y política de contraseña: mínimo 8 caracteres con una mayúscula, un número y un símbolo. **DoD:** Un usuario bloqueado no puede iniciar sesión hasta que el administrador lo active; las reglas de contraseña se validan en el backend. Pruebas en verde.	Dev Backend (Laura Pérez)
TSK-DEV2-103: RBAC en el API Gateway	HU-03 (3 SP) / RF-03	• Guard de autorización en el gateway que valida el JWT y los permisos contra la matriz de 5 roles (ADMIN_GENERAL, RRHH_OPERATIVO, JEFE_AREA, FUNCIONARIO, GERENCIA) antes de enrutar a cada servicio. • Propagación de id_usuario y rol a los servicios mediante cabeceras internas firmadas. **DoD:** Un rol sin permiso recibe 403 en cada endpoint protegido; la matriz está cubierta con pruebas automáticas por rol y acción.	Dev Backend (Laura Pérez) + Arquitecto (Axl Severich)
TSK-DEV2-104: ms-auth: registro de auditoría inmutable	HU-04 (3 SP) / RF-05 / RNF-06	• Consumidor de AUDITORIA_REGISTRADA e interceptor compartido que publica el evento en toda operación de escritura (POST, PUT, PATCH, DELETE) de cualquier servicio, con id_usuario, IP de origen, módulo, acción y payload_json (datos previos y posteriores). • GET /auditoria con filtros por usuario, módulo y fecha (paginado) para ADMIN_GENERAL y lectura para GERENCIA. **DoD:** Un cambio de sueldo y una baja aparecen en la auditoría con el detalle antes/después; los registros no se pueden modificar ni eliminar ni siquiera desde la API. Pruebas en verde.	Dev Backend (Laura Pérez)

Tarea	Historia / Origen	Descripción / Definición de Hecho (DoD)	Responsable
TSK-DEV2-105: ms-personal: áreas y cargos	HU-11 (3 SP) / RF-10	• Módulo catalogos de ms-personal: CRUD de áreas con jerarquía recursiva (área padre/hija), jefe de área y CRUD de cargos. • Validación: no se elimina un área o cargo con funcionarios asignados ni se crean ciclos en la jerarquía. Endpoint GET /areas/tree para el organigrama. **DoD:** Árbol de áreas devuelto correctamente con 3 niveles de profundidad; entregado en los primeros días porque el alta de funcionarios (TSK-DEV3-101) depende de este catálogo. Pruebas en verde.	Dev Backend (Laura Pérez)
TSK-DEV3-101: ms-personal: alta de funcionario e inamovilidad	HU-05 (5 SP) / RF-06 / RF-13 / RNF-05	• POST /employees registra datos personales, área, cargo, haber básico y fecha de ingreso; el estado inicia en EN_PRUEBA por 90 días. La CI y la remuneración se cifran con AES-256. • Carga obligatoria de la firma gráfica (.png/.jpeg) a Cloudinary con URL firmada de corta duración y registro de marcas de inamovilidad (gestación, progenitor, Ley 223). Validación del límite de 15% de personal extranjero. • El evento EMPLEADO_CREADO se registra en el Outbox en la misma transacción y se publica en RabbitMQ. **DoD:** El alta crea el funcionario, publica el evento y ms-vacaciones crea su saldo; sin firma no se permite el alta. Pruebas unitarias e integración en verde.	Dev Backend (Elias Lecoña)
TSK-DEV3-102: ms-personal: modificaciones, ratificación y listado	HU-06 (3 SP) / HU-08 / RF-07 / RF-09 / RF-13	• PATCH /employees/:id para cambiar área, cargo o remuneración conservando el historial laboral con fecha efectiva y motivo; publica EMPLEADO_MODIFICADO. • Acción de ratificación (EN_PRUEBA a RATIFICADO) una vez cumplidos los 90 días, solo para RRHH_OPERATIVO o ADMIN_GENERAL. • GET /employees con búsqueda por nombre o CI, filtros por área, cargo, estado e inamovilidad, y paginación (consumido por TSK-DEV1-104). **DoD:** Cada cambio queda en historial_laboral; la ratificación antes de 90 días es rechazada; el listado responde con p95 menor a 300 ms. Pruebas en verde.	Dev Backend (Elias Lecoña)
TSK-DEV3-103: ms-personal: baja de personal, bloqueo por inamovilidad y finiquito	HU-07 (8 SP) / RF-08 / RF-13	• POST /employees/:id/baja con fecha y motivo. Si existe una inamovilidad activa, la baja se bloquea hasta adjuntar la autorización del Ministerio de Trabajo (archivo en Cloudinary). • Cálculo del finiquito: Sueldo Promedio Indemnizable (promedio de los últimos 3 meses; mientras no existan boletas se usa el haber básico), indemnización por años y duodécimas, desahucio de 3 × SPI solo si aplica_desahucio es verdadero, aguinaldo proporcional y vacaciones pendientes. • Publica EMPLEADO_DADO_DE_BAJA, tras lo cual ms-vacaciones congela el saldo. **DoD:** Con inamovilidad activa la baja devuelve error y no cambia el estado; con autorización o sin fuero la baja se registra y el finiquito coincide con los casos calculados a mano (con y sin desahucio). Pruebas en verde.	Dev Backend (Elias Lecoña) (revisión: Arquitecto)

Tarea	Historia / Origen	Descripción / Definición de Hecho (DoD)	Responsable
TSK-VAC-101: ms-vacaciones: elegibilidad y saldo por gestión	HU-12 (3 SP) / RF-14 / RF-15	• Consumidor de EMPLEADO_CREADO, EMPLEADO_MODIFICADO y EMPLEADO_DADO_DE_BAJA que mantiene una copia local mínima del funcionario (empleado_ref). • Cálculo de la elegibilidad (estado RATIFICADO y antigüedad de 1 año o más) y del saldo: 15 días por gestión cumplida (parámetro configurable), con días asignados, tomados y pendientes por gestión. GET /vacaciones/saldo. **DoD:** Un funcionario en prueba o con menos de 1 año aparece como no elegible; uno elegible muestra 15 días por gestión; el saldo se crea al consumir EMPLEADO_CREADO. Pruebas en verde.	Scrum Master / Dev Backend (Jean Fernandez)
TSK-VAC-102: ms-vacaciones: solicitud de vacaciones	HU-13 (5 SP) / RF-16	• POST /vacaciones/solicitudes con rango de fechas. Validaciones: elegibilidad, saldo suficiente, cálculo de días hábiles descontando fines de semana y feriados oficiales, y ausencia de traslape con otras solicitudes o licencias. • La solicitud inicia en PENDIENTE_JEFE e identifica al jefe inmediato (id_jefe_area). **DoD:** Rangos que cruzan un feriado o un fin de semana descuentan los días correctos; una solicitud que traslapa o excede el saldo es rechazada con un mensaje claro. Pruebas en verde.	Scrum Master / Dev Backend (Jean Fernandez)
TSK-VAC-103: ms-vacaciones: aprobación en 2 niveles y ruta de excepción	HU-14 (5 SP) / RF-17	• Flujo PENDIENTE_JEFE a APROBADO_JEFE (Jefe Inmediato) y a APROBADO_FINAL (RRHH), con rechazo en cualquier nivel y descuento del saldo solo en la aprobación final. • Si dias_solicitados supera dias_pendientes, la solicitud estándar se bloquea y se habilita la Solicitud de Excepción derivada a Gerencia o ADMIN_GENERAL para su aprobación manual. Alerta GESTION_BLOQUEADA al acumular 2 gestiones pendientes. • Cada transición se registra en el historial y en la auditoría. **DoD:** Un jefe no puede aprobar solicitudes de otra área; la aprobación final descuenta el saldo; la excepción solo la pueden resolver los roles autorizados. Pruebas de integración en verde.	Scrum Master / Dev Backend (Jean Fernandez)
TSK-SCRUM-101: Sprint Planning y tablero de Jira del Sprint 1	Ceremonia Scrum	• Facilitar el Sprint Planning y cargar en Jira las 14 historias (HU-01 a HU-08, HU-10 a HU-14 y HT-04) con sus tareas, responsable, story points, prioridad y fechas. Acordar el orden de las dependencias: infraestructura y BD, contratos, auth, catálogos, personal, vacaciones y pantallas. **DoD:** Tablero de Jira con todas las tareas asignadas y el Sprint 1 iniciado.	Scrum Master (Jean Fernandez)
TSK-SCRUM-102: Dailies, seguimiento de impedimentos y burndown chart	Ceremonia Scrum	• Daily de 15 minutos con foco en bloqueos y dependencias entre servicios y frontend (contratos de API, eventos, uso de mocks mientras el backend no esté disponible). Registro de impedimentos con responsable y fecha de resolución, y actualización diaria del burndown. **DoD:** Registro de impedimentos al día y burndown actualizado diariamente en Jira.	Scrum Master (Jean Fernandez)

Tarea	Historia / Origen	Descripción / Definición de Hecho (DoD)	Responsable
TSK-SCRUM-103: Requerimientos y matriz de trazabilidad	Entregable de gestión / RF-01 a RF-17 / RF-30	• Documento de requerimientos (RF, RNF y reglas de negocio) y matriz de trazabilidad RF a HU a tarea, publicado en docs/requerimientos.md y actualizado cada sprint. **DoD:** Matriz publicada con cada tarea de este sprint enlazada a su historia y requerimiento.	Scrum Master (Jean Fernandez)
TSK-SCRUM-104: Colección de pruebas de API y variables de entorno	Soporte al equipo	• Colección compartida (Bruno o Postman) con los endpoints de auth, personal y vacaciones, y entornos local y de demo. Consolidar en .env.example las variables nuevas del sprint, sin valores reales. **DoD:** Colección disponible para todo el equipo antes del tercer día y .env.example actualizado en develop.	Scrum Master (Jean Fernandez)
TSK-SCRUM-105: Refinamiento del backlog del Sprint 2	Ceremonia Scrum	• Sesión de refinamiento con el Arquitecto para preparar las historias del Sprint 2 (HU-09, HU-15 a HU-26): criterios de aceptación, estimación y dependencias, incluyendo el resultado del spike de boletas. **DoD:** Historias del Sprint 2 refinadas y estimadas en Jira antes del cierre del Sprint 1.	Scrum Master (Jean Fernandez) + Arquitecto (Axl Severich)
TSK-SCRUM-106: Sprint Review y Retrospectiva del Sprint 1	Ceremonia Scrum	• Guion de demo: inicio de sesión con roles distintos; creación de un usuario y bloqueo tras 3 intentos fallidos; alta de un funcionario (EN_PRUEBA) con su evento y su saldo de vacaciones; intento de baja bloqueado por inamovilidad; ratificación; solicitud de vacaciones con aprobación en 2 niveles y una solicitud que excede el saldo con su excepción; organigrama; y registro de auditoría de los cambios realizados. Retrospectiva con acciones de mejora. **DoD:** Acta del Review y de la Retrospectiva en la carpeta de entregables; pendientes registrados en Jira.	Scrum Master (Jean Fernandez)

Entregable del sprint: sistema base ejecutándose en local con un solo comando, con identidad y roles funcionando, el módulo de Personal completo (alta, modificación, ratificación, baja con bloqueo por inamovilidad, organigrama y búsqueda) y el módulo de Vacaciones completo (saldo, solicitud y aprobación en 2 niveles con excepción), con eventos entre servicios y auditoría activa, cumpliendo menos de 300 ms (p95) en los endpoints principales.

Notas del Scrum Master: el sprint compromete 54 SP en una semana, por lo que el alcance es exigente; si hay riesgo, HU-10 (organigrama) y HU-04 (visor de auditoría) son las primeras candidatas a pasar al Sprint 2, porque el 70% depende de Personal y Vacaciones. Los contratos de TSK-ARQ-104 deben aprobarse hasta el 30/10 para que backend y frontend trabajen en paralelo; mientras un endpoint no esté disponible, el frontend usa datos simulados (mocks). Orden de dependencias: infraestructura y bases de datos (Axl) → auth y catálogos de áreas y cargos (Laura) → alta de funcionarios (Elias) → saldo y solicitudes de vacaciones (Jean), por lo que Laura entrega HU-11 en los primeros días. Jean combina el rol de Scrum Master con el backend de vacaciones sin dejar las ceremonias. Elias y Laura se revisan los PR de forma cruzada, y el Arquitecto revisa los PR críticos (auth, gateway, eventos y esquemas). El 1/11 es domingo (Todos Santos) y no reduce días hábiles.

2. Revisión de requerimientos e Historias de Usuario

Los requerimientos funcionales, no funcionales y las historias de usuario que se llevan a cabo en este Sprint son los siguientes:

ID	Requerimiento / Historia de usuario
RF-01	El sistema debe permitir iniciar sesión con usuario y contraseña cifrada (Argon2id) y emitir un token JWT firmado (RS256) con los roles y permisos del usuario.
RF-02	El sistema debe cerrar sesión e invalidar la sesión por tiempo de expiración, con manejo de Refresh Tokens.
RF-03	El sistema debe restringir endpoints y vistas del frontend según la matriz RBAC de 5 roles.
RF-04	El sistema debe permitir crear, activar, bloquear y administrar usuarios, asignando roles y permisos.
RF-05	El sistema debe registrar logs inmutables de auditoría para accesos y cambios sensibles.
RF-06	El sistema debe registrar el alta de un funcionario con datos personales, área, cargo, haber básico, fecha de ingreso, firma digitalizada e inamovilidad laboral.
RF-07	El sistema debe permitir modificar área, cargo y remuneración conservando el historial laboral con fecha efectiva y motivo.
RF-08	El sistema debe procesar la baja de un funcionario verificando el bloqueo por inamovilidad activa y generando la liquidación/finiquito.
RF-09	El sistema debe permitir listar, buscar y filtrar funcionarios por nombre, CI, área, cargo, estado e inamovilidad.
RF-10	El sistema debe permitir administrar los catálogos de áreas (jerarquía de árbol) y cargos.
RF-12	El sistema debe mostrar el organigrama interactivo por áreas, cargos y jefe de área.
RF-13	El sistema debe publicar eventos asíncronos de personal (EMPLEADO_CREADO, EMPLEADO_MODIFICADO, EMPLEADO_DADO_DE_BAJA) mediante RabbitMQ.
RF-14	El sistema debe determinar la elegibilidad para vacaciones (estado RATIFICADO y antigüedad de 1 año o más).
RF-15	El sistema debe calcular y controlar el saldo de vacaciones por gestión (15 días por año cumplido en el MVP).
RF-16	El sistema debe registrar solicitudes de vacaciones validando saldo, traslapes, días hábiles y feriados.
RF-17	El sistema debe procesar la aprobación en 2 niveles (Jefe Inmediato y RRHH) y derivar a excepción por Gerencia cuando se exceda el saldo.
RF-30	El sistema debe mantener sincronizados los datos básicos del funcionario entre servicios mediante eventos con RabbitMQ.
RNF-01	Respuestas de las APIs en menos de 300 ms y carga de pantallas en menos de 1.5 s.
RNF-04	Tokens JWT con RS256 (TTL de 15 a 30 minutos) y tráfico cifrado con TLS 1.3 / HTTPS.
RNF-05	Cifrado AES-256 en reposo para salarios, finiquitos y cédulas de identidad.
RNF-06	Registro inmutable de auditoría para los cambios sensibles.
RNF-08	Separación en 3 capas (Domain, Application, Infrastructure) sin dependencias externas en el core.

ID	Requerimiento / Historia de usuario
RNF-10	Interfaz responsive con Tailwind + shadcn/ui, modo claro/oscuro y accesibilidad WCAG 2.1 AA.
HU-01	Como usuario del sistema, quiero iniciar sesión con credenciales válidas, para acceder a las funciones según mi rol. (3 SP)
HU-02	Como Administrador General, quiero crear, activar o bloquear usuarios y asignar roles, para gestionar los accesos. (3 SP)
HU-03	Como sistema, quiero restringir las vistas y endpoints según el rol asignado, para garantizar la confidencialidad de la información. (3 SP)
HU-04	Como Auditor / Administrador, quiero consultar un log inmutable de operaciones sensibles, para cumplir con las normas de ASFI. (3 SP)
HU-05	Como Operador de RRHH, quiero registrar a un nuevo funcionario con sus datos, cargo y fuero laboral, para ingresarlo a la nómina de ARCA. (5 SP)
HU-06	Como Operador de RRHH, quiero actualizar el área, cargo o salario de un empleado, para mantener actualizado el expediente. (3 SP)
HU-07	Como Operador de RRHH, quiero registrar la desvinculación de un trabajador, para darlo de baja y generar su finiquito. (8 SP)
HU-08	Como usuario de RRHH, quiero buscar empleados por nombre, CI, área o estado, para agilizar la localización de expedientes. (2 SP)
HU-10	Como Gerente / RRHH, quiero visualizar el organigrama de la empresa, para entender la estructura jerárquica de las áreas. (3 SP)
HU-11	Como Administrador / RRHH, quiero crear y editar departamentos y puestos de trabajo, para estructurar la organización. (3 SP)
HU-12	Como sistema, quiero calcular la antigüedad y asignar 15 días por gestión cumplida, para establecer los saldos disponibles. (3 SP)
HU-13	Como Funcionario, quiero registrar una solicitud de vacaciones en un rango de fechas, para programar mi descanso anual. (5 SP)
HU-14	Como Jefe Inmediato / RRHH, quiero aprobar o rechazar solicitudes de vacaciones, para autorizar el goce del beneficio. (5 SP)
HT-04	Como sistema, quiero comunicar eventos de personal entre microservicios vía RabbitMQ, para mantener la consistencia de datos de forma asíncrona. (5 SP)